

Document number	P3207R0
Date	2024-03-24
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Library Evolution Working Group (LEWG)

More & like

Table of contents

- More & like
 - Abstract
 - Motivational Example
 - `function_ref`
 - `optional`
 - Where else and where not
 - Language feature
 - Summary
 - References

Abstract

Our new and existing reference types could be made to be more like references. This will reduce the misuse of these reference types and make it easier to reason about dangling.

Motivational Example

`function_ref`

A stripped down `function_ref` has been created to better illustrate the requested changes. This new version of `function_ref` has been renamed `function_ptr`. Its address of operators has been deleted. Further, an alias to a `const function_ptr` has been created and named `function_ref`.

given

```
#include <string>
#include <vector>
//#include <functional>

// NOTE: function_ref renamed to function_ptr since it is reseatable.
template< class >
class function_ptr;

template< class R, class... Args >
class function_ptr<R(Args...) noexcept>
{
public:
    function_ptr() = delete;
    constexpr function_ptr(const function_ptr& other) noexcept = default;
    constexpr function_ptr(function_ptr&& other) noexcept = default;
    constexpr function_ptr& operator=( const function_ptr& ) noexcept = default;
    constexpr function_ptr& operator=( function_ptr&& other ) noexcept = default;
    ~function_ptr() noexcept = default;
    // NOTE: The 'address of' operators were deleted to prevent
    // making a reference to a reference type
    // & & & & & = &
    // a reference to a reference type should just be a copy
    function_ptr* operator&(void) = delete;
    function_ptr const *operator&(void) const = delete;
    function_ptr volatile *operator&(void) volatile = delete;
    function_ptr const volatile *operator&(void) const volatile = delete;

    // MISSING; guaranteed efficiency for 2 pointer design
    constexpr function_ptr(void* s, R (*f)(void* state, Args...) noexcept) noexcept
        : state{s}, thunk{f} {}

    R operator()( Args... args ) const noexcept
    {
        return thunk(state, std::forward<Args>(args)...);
    }
private:
    void* state;
    R (*thunk)(void* state, Args...) noexcept;
};

// NOTE: reference types should by default be const to make it more & like
template< class T >
using function_ref = const function_ptr<T>;
```

So what do these minor changes buy us.

1. By deleting the address of operators, we ensure proper usage of the reference type. Reference types are meant to be passed by copy not by reference. This removes additional avenues of dangling. This is consistent with references in general. If we retained these operators than they should return the address of the referent instead of the address of the reference type.
2. Lvalue references are immutable. "A reference is similar to a const pointer such as `int* const p` (as opposed to a pointer to const such as `const int* p`)" ^[1] References and constant pointers are easier to reason about dangling. Since references are not reseatable and constant pointers are not by default reseatable, they are just aliases to the referent that they point to. This means at compile time, once they point to a global, local or temporary, they always point to said global, local or temporary. Further since the constness is baked in, a programmer is less likely to forget to add it.

```
int main()
{
    function_ptr<int (int) noexcept> fp{nullptr,
        [](void* state, int i) noexcept { return test(i); }};
    fp = {nullptr, [](void* state, int i) noexcept { return test(i + 1); }};
    function_ref<int (int) noexcept> fr{nullptr,
        [](void* state, int i) noexcept { return test(i); }};
    //auto a = &fr; // deleted because function_ref is a reference type
    // & & & & = &
    // shouldn't reassign function_ref to make it more like ref
    //fr = {nullptr, [](void* state, int i) noexcept { return test(i + 1); }};
    // NOTE: function_ptr can still be created from function_ref
    // ensuring its use in collections
    fp = fr;
    std::vector<function_ptr<int (int) noexcept>> fps;
    //std::vector<function_ref<int (int) noexcept>> frs; // correct compiler error

    return 0;
}
```

optional

It has been proposed to allow `std::optional` to support being a pure reference type: `std::optional<T&>` ^[2]. While one must pause to decide whether this specialization should have its address of operators deleted since `std::optional` is an existing type, there shouldn't be any impediment to adding one alias to bake constness in as a convenience. This

is especially important as there are plenty of methods in the standard library that currently return a reference with undefined behavior which would be better served by returning a `const optional<T&>` with defined behavior. Making sure to add `const` to our `optional<T&>` would also be important for our future proposals such as Better lookups for `map` and `unordered_map` ^[3].

given

```
#include <vector>
#include <optional> // plus optional<&>

// const baked in by default
template<class T>
using optional_ref = const optional<T&>; // OR opt_ref
```

Here the proposed `std::optional<T&>` ^[2:1] behaves correctly like a pointer with a constant pointer able to be implicitly converted to a non constant pointer.

```

int main()
{
    int value = 0;
    int * p00 = &value;
    int const * p10 = &value;
    int * const p01 = &value;
    int const * const p11 = &value;
    p00 = p00;
    //p00 = p10;// error
    p00 = p01;
    //p00 = p11;// error
    p10 = p00;
    p10 = p10;
    p10 = p01;
    p10 = p11;
    //p01 = p00;// error
    //p01 = p10;// error
    //p01 = p01;// error
    //p01 = p11;// error
    //p11 = p00;// error
    //p11 = p10;// error
    //p11 = p01;// error
    //p11 = p11;// error
    return 0;
}

```

```

int main()
{
    int value = 0;
    optional<int&> op00{value};
    optional<const int&> op10{value};
    const optional<int&> op01{value};
    const optional<const int&> op11{value};
    op00 = op00;
    //op00 = op10;// error
    op00 = op01;
    //op00 = op11;// error
    op10 = op00;
    op10 = op10;
    op10 = op01;
    op10 = op11;
    //op01 = op00;// error
    //op01 = op10;// error
    //op01 = op01;// error
    //op01 = op11;// error
    //op11 = op00;// error
    //op11 = op10;// error
    //op11 = op01;// error
    //op11 = op11;// error
    return 0;
}

```

The `optional_ref` alias similarly behaves correctly but this time `const` is baked in by default.

```

int main()
{
    int value = 0;
    optional<int&> op00{value};
    optional<const int&> op10{value};
    const optional<int&> op01{value};
    const optional<const int&> op11{value};
    op00 = op00;
    //op00 = op10;// error
    op00 = op01;
    //op00 = op11;// error
    op10 = op00;
    op10 = op10;
    op10 = op01;
    op10 = op11;
    //op01 = op00;// error
    //op01 = op10;// error
    //op01 = op01;// error
    //op01 = op11;// error
    //op11 = op00;// error
    //op11 = op10;// error
    //op11 = op01;// error
    //op11 = op11;// error
    return 0;
}

```

```

int main()
{
    int value = 0;
    optional<int&> op00{value};
    optional<const int&> op10{
optional_ref<int> op01{val
optional_ref<const int> op
    op00 = op00;
    //op00 = op10;// error
    op00 = op01;
    //op00 = op11;// error
    op10 = op00;
    op10 = op10;
    op10 = op01;
    op10 = op11;
    //op01 = op00;// error
    //op01 = op10;// error
    //op01 = op01;// error
    //op01 = op11;// error
    //op11 = op00;// error
    //op11 = op10;// error
    //op11 = op01;// error
    //op11 = op11;// error
    return 0;
}

```

Where else and where not

Similar changes could be performed on existing pure reference types in the standard library such as `std::span`, `std::string_view` and `std::mdspan`. These changes would not be recommended for hybrid reference types such as `std::tuple` and `std::variant` should reference support be added.

Language feature?

This could better be implemented as a language feature. I would be glad to write such a proposal, if there was sufficient interest.

```
[[const_by_default]]
class function_ref;

// usage
function_ref fr1;// const by default
mutable function_ref fr2;// const has been disengaged
```

Summary

Deleting address of operators from pure reference types minimizes the misuse of said types and reduces the avenues to dangling. Ensuring pure reference types are immutable by default makes it easier for programmers to reason about the lifetimes of the referents that these reference type instances points to.

References

1. <https://isocpp.org/wiki/faq/references#reseating-refs>
(<https://isocpp.org/wiki/faq/references#reseating-refs>) 
2. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2988r3.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2988r3.pdf>)  
3. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3091r0.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3091r0.html>) 